The BJKST Algorithm

Counting Distinct Elements in a Data Stream

Manvi G. and Nehal S.

Indian Statistical Institute

1. What problem are we solving?

We watch a huge stream of data:

$$a_1,\ a_2,\ a_3,\ \ldots,\ a_N$$

We do not care how many times each element appears.

We only ask: **How many different elements have appeared?**

1. What problem are we solving?

Example stream:

$$101,\ 205,\ 101,\ 310,\ 205,\ 450,\ 520,\ 310$$

Distinct elements in our example:

$$\{101,\ 205,\ 310,\ 450,\ 520\}$$

So the number of distinct elements is

$$d = 5.$$

What is d

d denotes the total number of unique (distinct) elements seen in the stream.

2. The naive method

Naive approach

Keep a set $S = \emptyset$. For every incoming element a_i:

- if $a_i \notin S$, put it into S
- if $a_i \in S$, do nothing

At the end, $|S| = d$.

2. The naive method on our example

Element	**Set S afterwards**
101	$\{101\}$
205	$\{101, 205\}$
101	unchanged
310	$\{101, 205, 310\}$
205	unchanged
450	$\{101, 205, 310, 450\}$
520	$\{101, 205, 310, 450, 520\}$

Answer: **5**. It works!

2. So what is the problem?

Imagine a website receiving billions of events.

- Hundreds of millions of distinct users, IP addresses or search queries
- We would have to store hundreds of millions of values

The problem

Exact counting can need **enormous memory**.

The streaming idea

Don't store everything. Keep a very small summary and **estimate** the answer.

3. Two different questions

Exact question

Tell me exactly how many distinct elements occurred.

Approximate question

Give me a number that is very close to the true number, with a controlled chance of being wrong.

BJKST solves the second problem.

4. It sounds strange. . .

Suppose there are 1,000,000,000 possible elements.

We do not want to store all of them.

Idea: randomly pick only 1% of the distinct elements.

4. A sample can tell us the total

Say we end up with 10,000,000 selected elements. Then

$$d \approx \frac{10{,}000{,}000}{0.01} = 1{,}000{,}000{,}000$$

Basic statistical idea

Instead of remembering everything, remember a **representative sample**.

But: how do we pick the sample without first storing a gigantic set?
The answer is **hashing**.

5. A hash function

Instead of using the elements directly, we use a hash function $h(x)$.



We treat it as a **black box**. We only need its statistical properties.

5. Our hash function

We have

$$h : \{1, 2, \ldots, m\} \to \{1, 2, \ldots, M\}$$

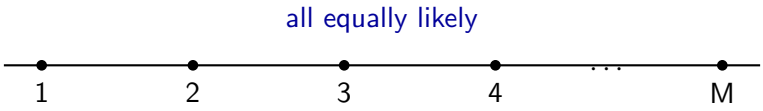
We choose h at random from a **pairwise-independent** hash family.

We need only three properties.

5. Property 1: Uniformity

For any input x, $h(x)$ is uniformly distributed over $\{1, 2, \ldots, M\}$

So every hash value is equally likely.



5. Property 2: Pairwise independence

For two different elements $x \neq y$, their hash values behave independently enough for our calculations.

In particular,

$$P\big(h(x) = h(y)\big) = \frac{1}{M}$$

This is the important collision property.

5. Property 3: Same input, same hash

Once h is chosen, a fixed input x **always** gives the same value.

Example: if $h(101) = 347$, then every occurrence of 101 gives 347.

Why this matters

Repeated copies of the same element are **not** counted as new distinct elements.

5. Summary of the black box

We do not explain how h is built inside.

We simply say:

> *The hash function is a black box whose outputs are* **uniform** *and* **pairwise independent**.

That is exactly the information the algorithm needs.

6. Hashing makes a sample for free

Suppose we say:

Keep an element only if its hash value is very small.

For example: keep x if $h(x) \leq 100$.

6. How many do we keep?

Say hash values are uniform over $1, \ldots, 1{,}000{,}000$.

Then the fraction of elements we keep is about

$$\frac{100}{1{,}000{,}000} = 0.01\%$$

Key point

We got a random-looking sample using only the hash value.
We do not need to select elements randomly one by one. **The hash function does the selection for us.**

7. Setting up

Choose a number k. We will keep only the k **smallest** hash values seen so far.

BJKST uses

$$M = m^3$$

and a pairwise-independent hash function

$$h : \{1, \ldots, m\} \to \{1, \ldots, M\}$$

Why $M = m^3$?

A large range keeps hash collisions unlikely. We will see exactly why m^3 is a good choice in section 16.

7. The algorithm

1. Choose the random hash function h.
2. Process the stream one element at a time. For each a_i, compute $h(a_i)$.
3. Keep only the k smallest hash values.
4. Let z_k be the k-th smallest value currently stored.
5. Output

$$\boxed{\hat{d} = \frac{k\,(M+1)}{z_k}}$$

as the estimate of the number of distinct elements.

Now to understand why this formula works, let's view the algorithm step-wise.

8. More distinct values, smaller hash values

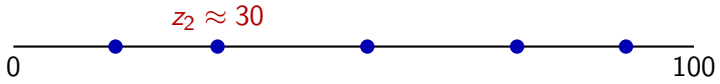
Suppose there are d distinct elements and we hash all of them into range $\{1, \ldots, M\}$

- If d is **large**: we will see some very small hash values.
- If d is **small**: the smallest hash values tend to be larger.

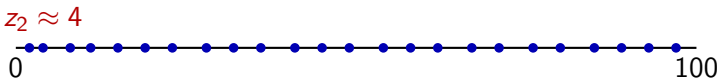
Let's see an example to visualize this

8. A simple picture: $M = 100$

Case A: only 5 random points



Case B: 50 random points



8. What the picture says

more points $\Rightarrow$ smaller k-th smallest value

fewer points $\Rightarrow$ larger k-th smallest value

This means that the smallest value give us an idea of number of distinct elements in the stream

9. Why not just use the absolute minimum?

Our first instinct might be to track just the single smallest value ($h_{\min}$)

However, suppose that the true number of distinct elements is 1000.

In one experiment we might get

$$h_{\min} = 4,$$

and in another

$$h_{\min} = 30$$

That is a lot of randomness!

Therefore, using just the absolute minimum is far too noisy—**we need a fix to make our estimate reliable.**

9. The fix: keep several small values

Instead of only the smallest hash value, keep the k smallest:

$$h_1 < h_2 < h_3 < \cdots < h_k$$

Then use the k-th smallest value, z_k.

The key idea of the first BJKST formulation

Hash into a large range, keep the k smallest hashed values.

Next: how do we turn z_k into an estimate of d?

10. Start with discrete uniform hash values

Suppose there are d distinct elements and hash values lie in

$$\{1, 2, \ldots, M\}$$

To derive the expectation, we first consider the idealized model in which d different values are selected uniformly **without replacement** from these M integers.

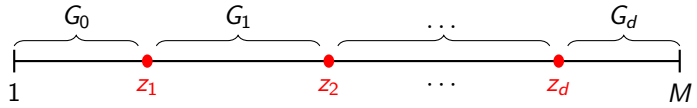
Write their ordered values as

$$z_1 < z_2 < \cdots < z_d$$

We want to find $E[z_k]$

10. The gaps $G_0, G_1, \ldots, G_d$ and their expectation

The d selected points divide the range $\{1, \ldots, M\}$ into $d+1$ gaps:



- **Gap sizes:** $G_0 = z_1 - 1, \quad G_i = z_{i+1} - z_i - 1, \quad G_d = M - z_d$
- These gaps contain all unselected integers:

$$G_0 + G_1 + \cdots + G_d = M - d$$

- By symmetry, all $d+1$ gaps share the same expectation:

$$E[G_i] = \frac{M-d}{d+1}$$

10. From the expectation to the estimator

The expectation shows the scale of the k-th smallest value:

$$E[z_k] = \frac{k(M+1)}{d+1}$$

Rearranging gives

$$d + 1 = \frac{k(M+1)}{E[z_k]}$$

This motivates estimating the distinct count from the observed z_k:

$$\boxed{\hat{d} = \frac{k(M+1)}{z_k}}$$

10. From the expectation to the estimator

The expectation shows the scale of the k-th smallest value:

$$E[z_k] = \frac{k(M+1)}{d+1}$$

Rearranging gives

$$d + 1 = \frac{k(M+1)}{E[z_k]}$$

This motivates estimating the distinct count from the observed z_k:

$$\boxed{\hat{d} = \frac{k(M+1)}{z_k}}$$

- We replace the theoretical expectation $E[z_k]$ with our actual observed sample z_k to estimate d.
- Because z_k is from a random sample, $\hat{d}$ acts as an estimate rather than the exact count.
- *Note: Our expectation proof assumed no hash collisions; in practice, M is chosen large enough so collisions are rare (we will see the formal collision probability bound on M later).*

11. The same idea here

Instead of relying on z_1, we use z_k.

Increasing k gives us more information and reduces the relative randomness of the estimate.

The standard analysis chooses k proportional to $1/\epsilon^2$, to get a desired relative accuracy ϵ.

11. How should we choose k?

The value of k depends on the relative error ϵ we want and on the probability guarantee required by the analysis.

The general accuracy scaling is

$$\boxed{k = O\left(\frac{1}{\epsilon^2}\right)}$$

12. What does k do?

- $k = 1$: very little information. We rely on only one extreme observation.
- $k = 100$: much more information about where the lower part of the hash distribution lies.

Therefore increasing k generally improves accuracy, but requires more memory.

Trade-off

larger $k \rightarrow$ more memory $\rightarrow$ better accuracy.

12. Why k smallest values instead of all values?

Suppose there are 10^9 distinct elements.

Storing all 10^9 hash values is still huge.

If our required accuracy allows $k = 1000$, we only maintain about 1000 hash values:

$$10^9 \longrightarrow 1000.$$

This is a massive reduction in stored information. That is the main streaming-algorithm idea.

12. What does ϵ mean?

We want our estimate $\hat{d}$ to have relative error at most ϵ:

$$\frac{|\hat{d} - d|}{d} \leq \epsilon \iff (1 - \epsilon)d \leq \hat{d} \leq (1 + \epsilon)d$$

Example: If $d = 1000$ and we set $\epsilon = 0.1$ (10% relative error):

$$900 \leq \hat{d} \leq 1100$$

- Smaller ϵ means a tighter target interval around the true count d.

12. The (ϵ, δ) guarantee

Since the algorithm uses randomized hashing, the guarantee holds with high probability rather than certainty:

$$\mathbb{P}\Big(|\hat{d} - d| \leq \epsilon d\Big) \geq 1 - \delta$$

There are two user-specified parameters:

- ϵ **(accuracy):** how much relative error is tolerated.
- δ **(confidence):** probability that the estimate falls outside the bound.

Example: For $\epsilon = 0.1$ and $\delta = 0.05$, the estimate is within 10% of d with probability at least 95%.

13. How each quantity affects the estimate

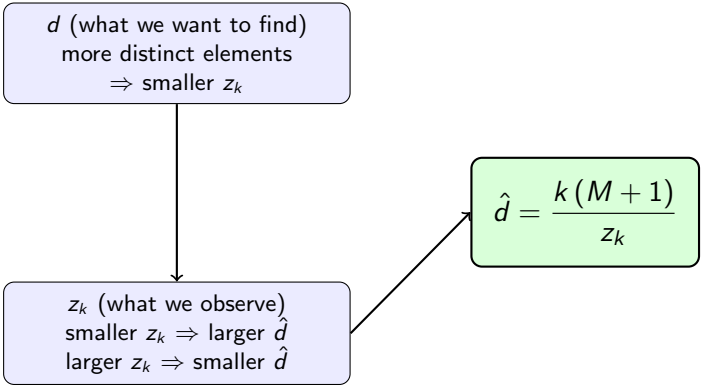
$$\hat{d} = \frac{k\,(M+1)}{z_k}$$

13. How each quantity affects the estimate

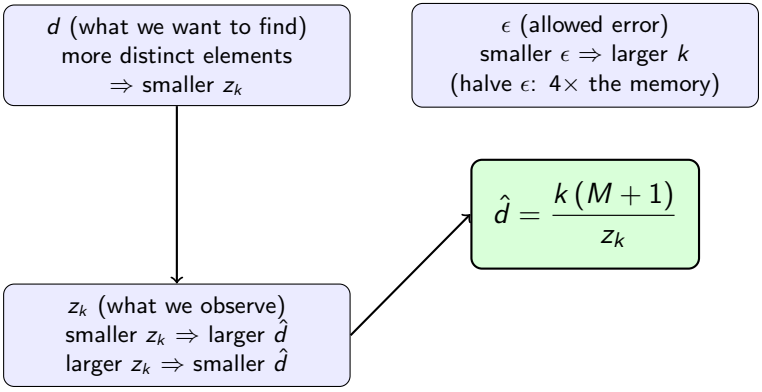
d (what we want to find)
more distinct elements
$\Rightarrow$ smaller z_k

$$\hat{d} = \frac{k\,(M+1)}{z_k}$$

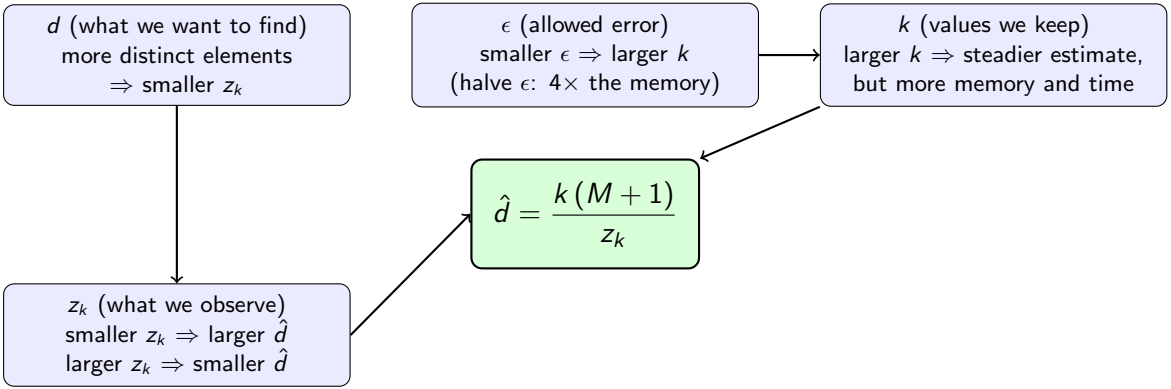
13. How each quantity affects the estimate



13. How each quantity affects the estimate

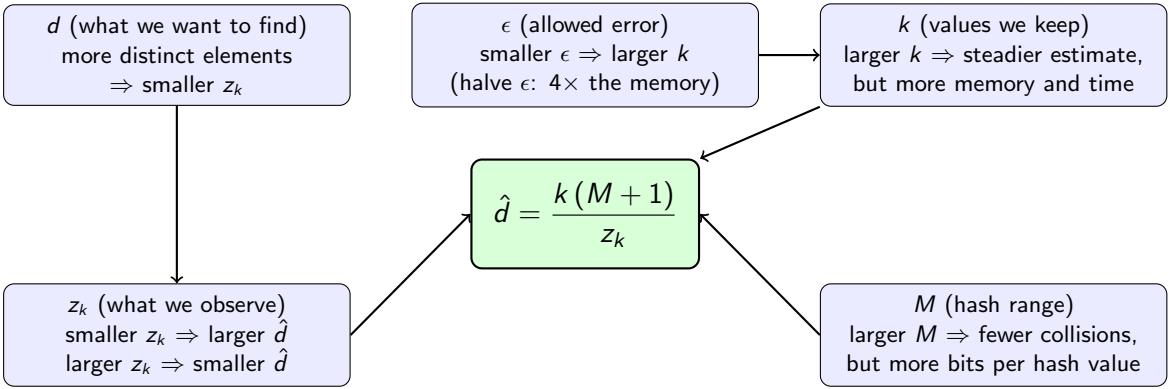


13. How each quantity affects the estimate



Note: $z_k \approx k(M+1)/d$, so a larger k also makes z_k larger. The estimate stays close to d, but with less noise.

13. How each quantity affects the estimate



Note: $z_k \approx k(M+1)/d$, so a larger k also makes z_k larger. The estimate stays close to d, but with less noise.

How unlikely are collisions, exactly?

14. A stream with repeats

Suppose our stream is

$$A,\ B,\ A,\ C,\ B,\ D,\ A$$

There are only **four** distinct elements:

$$A,\ B,\ C,\ D$$

14. What happens when we hash

When we hash, we compute

$$h(A),\ h(B),\ h(A),\ h(C),\ h(B),\ h(D),\ h(A)$$

- The repeated A's always produce the **same** hash value.
- The repeated B's always produce the **same** hash value.

Therefore, the distinct hash values correspond to the distinct elements, provided collisions between different elements are avoided.

This is why the hash-function properties are important.

14. Addressing collisions

A collision occurs when

$$x \neq y \quad \text{but} \quad h(x) = h(y)$$

We want collisions to be very unlikely.

14. A large range keeps collisions rare

BJKST chooses a large hash range:

$$M = m^3$$

For distinct x, y,

$$P\big(h(x) = h(y)\big) =$$

14. A large range keeps collisions rare

BJKST chooses a large hash range:

$$M = m^3$$

For distinct x, y,

$$P\big(h(x) = h(y)\big) = \frac{1}{M}$$

Because M is very large, collisions become unlikely.

(Why exactly m^3? We answer this in section 15, after we have the collision bound.)

15. Where Markov's inequality fits

We use it to bound the probability that **at least one hash collision occurs**.

This is a nice probability application.

15. Define a random variable

Suppose there are d distinct elements. Define

$$C = \text{number of colliding pairs}$$

Clearly,

$$C \geq 0$$

15. One indicator for each pair

For each pair $i < j$, define

$$I_{ij} = \begin{cases} 1, & h(x_i) = h(x_j) \\ 0, & \text{otherwise.} \end{cases}$$

Then

$$C = \sum_{i<j} I_{ij}$$

15. The expected number of colliding pairs

Since $P\big(h(x_i) = h(x_j)\big) = \dfrac{1}{M}$, we have

$$E[I_{ij}] =$$

15. The expected number of colliding pairs

Since $P\big(h(x_i) = h(x_j)\big) = \frac{1}{M}$, we have

$$E[I_{ij}] = \frac{1}{M}$$

There are $\binom{d}{2}$ pairs.
Let's see why.

15.1. Why do we have $\binom{d}{2}$ pairs?

Suppose we have a stream with d distinct elements $x_1, x_2, \ldots, x_d$

A hash collision happens when two distinct elements get the same hash value:

$$h(x_i) = h(x_j), \qquad i \neq j$$

15. 1. Why do we consider pairs?

The Pairwise Rule:
A collision is an event involving at least two elements.
If three or more elements collide (e.g., $h(x_1) = h(x_2) = h(x_3)$), all underlying pairs

$$(x_1, x_2), \quad (x_1, x_3), \quad (x_2, x_3)$$

also collide.
Every single multi-element collision breaks down into a specific number of pairs $\rightarrow$ keeping the total expected number of pairs extremely low mathematically guarantees that large collision clusters are also tightly bounded.

Conclusion: Bounding pair collisions is both necessary and sufficient to account for all collisions.

15. 2. Why are the pairs unordered?

The pair (x_i, x_j) is the same collision event as (x_j, x_i).

So we should **not count it twice**.

Therefore, we use **combinations** rather than permutations.

15. 3. Why $\binom{d}{2}$?

We have d elements and want to choose 2 at a time. The number of unordered pairs is

$$\binom{d}{2} = \frac{d!}{2!\,(d-2)!} = \frac{d(d-1)}{2}$$

Example: if $d = 4$,

$$\binom{4}{2} = \frac{4 \times 3}{2} = 6$$

The six pairs are

$$(x_1, x_2),\ (x_1, x_3),\ (x_1, x_4),\ (x_2, x_3),\ (x_2, x_4),\ (x_3, x_4)$$

15. Indicators and linearity of expectation

Recall, for each pair i, j:

$$I_{ij} = \begin{cases} 1, & \text{collision occurs,} \\ 0, & \text{otherwise,} \end{cases} \qquad E[I_{ij}] = P(I_{ij} = 1) = \frac{1}{M}$$

And because $C = \sum I_{ij}$, linearity of expectation gives

$$E[C] = \sum E[I_{ij}] = \frac{\binom{d}{2}}{M}$$

15. Markov's inequality

Markov's inequality

If $X \geq 0$, then for $a > 0$,

$$P(X \geq a) \leq \frac{E[X]}{a}$$

Here $X = C$, and we want the probability of **at least one collision**. That means $C \geq 1$

15. Apply Markov's inequality

Taking $a = 1$,

$$P(C \geq 1) \leq \frac{E[C]}{1}$$

Hence

$$P(C \geq 1) \leq \frac{\binom{d}{2}}{M}$$

So

$$\boxed{P(\text{at least one collision}) \leq \frac{\binom{d}{2}}{M}}$$

15. The bound when $M = m^3$

Suppose we choose $M = m^3$. If the number of possible elements is m, then even in the worst case $d \leq m$. Thus

$$P(\text{collision}) \leq \frac{\binom{m}{2}}{m^3}$$

Since $\binom{m}{2} = \frac{m(m-1)}{2}$,

$$P(\text{collision}) \leq \frac{m(m-1)}{2m^3} \quad \text{and therefore} \quad P(\text{collision}) < \frac{1}{2m}$$

15. A quick check with numbers

If $m = 1000$ possible elements, then

$$P(\text{collision}) < \frac{1}{2 \times 1000} = 0.0005$$

So for large m, this probability is very small.

We can safely assume hash values uniquely identify our elements without worrying about accidental overlaps.

15. Is m^3 a good choice? We try other powers of m

Worst case: $d = m$. For $M = m^c$ the collision bound becomes

$$P(\text{collision}) \leq \frac{m(m-1)}{2M} = \frac{m-1}{2\,m^{c-1}}$$

M	bound	value for $m = 100$	bits per hash value
m	$\frac{m-1}{2}$	49.5 (> 1, pointless)	7
m^2	$< \frac{1}{2}$	0.495	14
m^3	$< \frac{1}{2m}$	0.00495	20
m^4	$< \frac{1}{2m^2}$	0.0000495	27
m^5	$< \frac{1}{2m^3}$	0.000000495	34

Bits per hash value $= \lceil \log_2 M \rceil$

15. So why does BJKST use $M = m^3$?

- $M = m$: the bound is bigger than 1. It tells us nothing.
- $M = m^2$: the bound stays near $\frac{1}{2}$, however large m is.
- $M = m^3$: the bound is below $\frac{1}{2m}$, so it **shrinks** as m grows. This is the smallest whole-number power that does this.
- $M = m^4, m^5, \ldots$: even smaller bounds, but each extra power adds $\log_2 m$ bits to every stored hash value.

Conclusion

m^3 is a good balance between few collisions and memory. It is **not the only** choice: m^4 or m^5 would also work.

A short example: the stream A, B, C, A

Stream: $A,\ B,\ C,\ A$ (so $d = 3$)

Settings: $M = 100,\quad k = 2$ (keep only the 2 smallest hash values)

Suppose the random hash function gives

$$h(A) = 5, \qquad h(B) = 23, \qquad h(C) = 33$$

In real life M is huge, but the steps are exactly the same.

The stream, one element at a time

Element	Hash	2 smallest kept	What happens
A	5	$\{5\}$	there is room, so keep it
B	23	$\{23, 5\}$	there is room, so keep it
C	33	$\{23, 5\}$	33 is bigger than both: drop it
A	5 again	$\{23, 5\}$	already stored: nothing changes

The estimate

The stored values are 23 and 5, so $z_2 = 23$ (the 2nd smallest). Then

$$\hat{d} = \frac{k\,(M+1)}{z_k} = \frac{2 \times 101}{23} \approx 8.78$$

The true answer is $d = 3$.

What we saw

- We stored only 2 numbers, never the elements themselves.
- The repeated A changed nothing.
- A different random hash would give a slightly different estimate.

16. Unique IP addresses visiting a server

Now let's see a practical example of the project.

Imagine a busy web server. During one day it receives

$$1{,}000{,}000{,}000$$

requests.

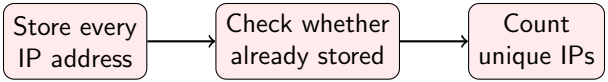
The administrator wants to know:

How many different IP addresses contacted the server today?

The stream could look like

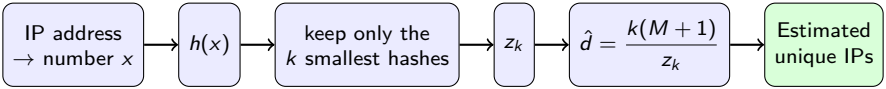
`203.0.113.5`, `198.51.100.7`, `203.0.113.5`, `192.0.2.44`, ...

16. The naive approach



But if millions of different IP addresses appear, the set needs a lot of memory.

16. The BJKST approach



An IP address $a.b.c.d$ is four numbers, so it is easy to turn into one number in base 256:

$$x = a \cdot 256^3 + b \cdot 256^2 + c \cdot 256 + d + 1$$

The administrator gets an approximate count **without storing every IP address**.

16. Other examples

Databases

How many different cities appear in a customer table with billions of rows? Hash every city name and keep the k smallest hash values.

Search terms (text)

The stream "weather", "maps", "weather", "news", "maps", "music" has **four** distinct terms. A search service can estimate this count for billions of queries.

An online shop (objects)

The stream apple, banana, phone, apple, ... How many different products were sold today?

What the experiment shows

- **Memory:** with $k = 1000$ we store 1000 numbers instead of 99,995 addresses, and the error is only a few percent.
- **Accuracy:** larger k gives smaller errors.
- **Cost:** larger k means more memory and more time. This is the trade-off.

17. Advantages

1. **Low memory.** We don't store the entire stream. We store only k hash values (each about $3 \log_2 m$ bits when $M = m^3$).
2. **One-pass streaming.** We can process elements as they arrive. We don't need to repeatedly scan the entire data.
3. **Works for enormous streams.** The algorithm is specifically designed for massive data streams.

17. Advantages

4. **Probability gives a guarantee.** We can mathematically analyze the error rather than simply saying the estimate "usually works."
5. **No need to know the actual identities after hashing.** The algorithm works with the hash representation.

18. Limitations

1. **It is approximate.** We don't necessarily get $\hat{d} = d$. We get an estimate.
2. **Randomness is involved.** Different choices of the hash function can produce slightly different estimates.

18. Limitations

3. **Hash collisions are possible.** We make them very unlikely by choosing a large hash range.
4. **Accuracy costs memory.** To get smaller relative error ϵ, we need roughly

$$k = O\left(\frac{1}{\epsilon^2}\right)$$

So higher accuracy requires more stored hash values.

References


Z. Bar-Yossef, T. S. Jayram, R. Kumar, D. Sivakumar and L. Trevisan. *Counting Distinct Elements in a Data Stream*. In *Randomization and Approximation Techniques in Computer Science (RANDOM 2002)*, LNCS 2483, Springer, 2002, pp. 1–10.


M. R. Salavatipour (lecturer), B. Fuller (scribe). *CMPUT 675: Algorithms for Streaming and Big Data, Lecture 3 (Sep 11, 2019): BJKST Algorithm and AMS $F_k(\sigma)$ Estimator*. Lecture notes, Fall 2019. Provided by our professor.


Anthropic. Claude. Used to draft the Beamer (LaTeX) slide code, to write and test the Python demonstrations.

Thank you

Questions?